Practical No. 4

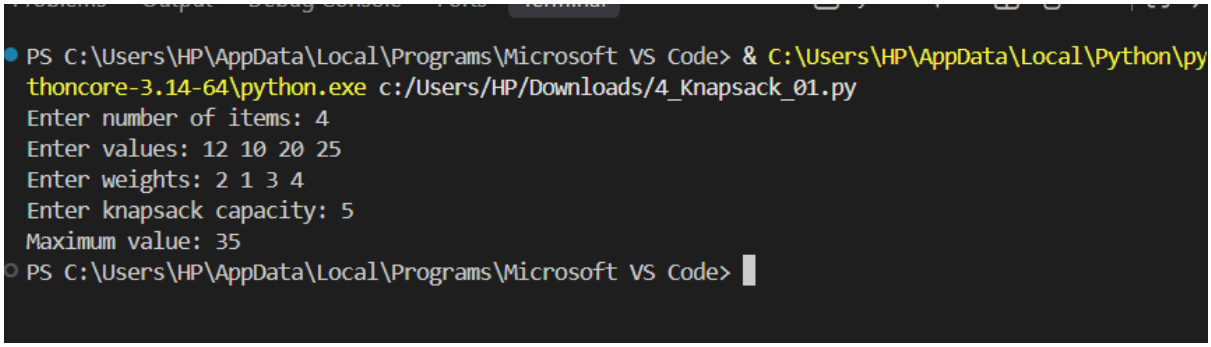
Title: Write a program to solve a 0-1 Knapsack problem using dynamic programming or branch and bound strategy.

```
def knapsack(W, wt, val, n):
  dp = [[0] * (W + 1) for _ in range(n + 1)]

  for i in range(1, n + 1):
    for w in range(1, W + 1):
      if wt[i - 1] <= w:
        dp[i][w] = max(
          val[i - 1] + dp[i - 1][w - wt[i - 1]],
          dp[i - 1][w]
        )
      else:
        dp[i][w] = dp[i - 1][w]

  return dp[n][W]

n = int(input("Enter number of items: "))
val = list(map(int, input("Enter values: ").split()))
wt = list(map(int, input("Enter weights: ").split()))
W = int(input("Enter knapsack capacity: "))

print("Maximum value:", knapsack(W, wt, val, n))
```

Output: